

DEBRE MARKOS UNIVERSTY

Institute of Technology

Internship Hosting Company: Dallol Tech PLC

STUDENT NAME	ID
1.ABEBE MESFIN	153689
2.ABRHAM SEMA	153719
3.ABRHAM TESFAYE	1304848

INTERNSHIP DURATION: MARCH 2 TO JUNE 2

SUBMITTED TO: BAYE ATINAFU (SUPERVISOR)

DATE OF SUBMISSION: 7/10/2026

DECLARATION

We hereby declare that this internship report is our own original work, prepared under the supervision of our mentor at Dallol Tech PLC and our academic supervisor at Debre Markos University. All sources of information used in the preparation of this report have been duly acknowledged.

We further declare that:

1. The work described in this report was carried out during our internship at Dallol Tech PLC from March 2 to June 2, as part of the Bachelor of Science in Software Engineering program at Debre Markos University, Institute of Technology.
2. The projects, tasks, and technical solutions presented in this report, namely the hotel booking mobile application, the hotel booking website, the ERP system, and the Oromia Communication Bureau platform, describe work in which we took part during the internship.
3. All documentation, libraries, websites, and other sources of information that we used are listed in the References section, and the ideas and work of others have been acknowledged.
4. We have respected the confidentiality of Dallol Tech PLC and its clients, and this report does not disclose passwords, private keys, or personal data belonging to customers.
5. This report has not been submitted, in whole or in part, to any other university or institution for any other academic award.
6. We understand that presenting the work of others as our own is a violation of academic integrity and may lead to disciplinary action by the university.

We take full responsibility for the honesty and accuracy of this report. The signatures below confirm that the student accepts this declaration and that the mentor at Dallol Tech PLC and the academic supervisor have reviewed the report.

ACKNOWLEDGEMENTS

First and foremost, we would like to express our sincere gratitude to the Almighty God for giving us the strength and wisdom to complete our internship successfully.

We are deeply grateful to the management and staff of **Dallol Tech PLC**, particularly to **Mr. Abel Mulugeta** (Founder & CEO), for providing us with the opportunity to intern at such a dynamic and innovative technology company. We would like to thank our mentor and the entire development team — including the senior developers, QA team, and project managers — for their continuous guidance, patience, and support throughout our internship period.

We would also like to extend our heartfelt thanks to our academic supervisor at **Debre Markos University, Institute of Technology**, for their valuable advice and direction throughout the internship program.

Finally, we thank our family and friends for their unwavering encouragement and moral support during the entire internship journey.

We also remain thankful to the instructors of the Software Engineering department at Debre Markos University, Institute of Technology. The courses we took in database systems, web programming, software engineering, object-oriented programming, data structures and algorithms, and human-computer interaction gave us the foundation we relied on every day at Dallol Tech PLC. Our thanks go as well to the fellow interns who shared ideas, tested features, and made the working environment friendly and productive.

EXECUTIVE SUMMARY

This report documents our internship experience at **Dallol Tech PLC**, a software and IT solutions company based in Addis Ababa, Ethiopia. The internship was conducted as part of the Bachelor of Science in Software Engineering program at Debre Markos University, Institute of Technology.

During our internship at Dallol Tech PLC, we worked on four major software projects that span both web and mobile platforms:

1. **Hotel Booking Mobile Application** — A Flutter-based mobile application for booking hotels across Ethiopia, featuring hotel search, room selection, payment integration, and user authentication.
2. **Hotel Booking Website** — A full-stack web application built with CodeIgniter 4 (PHP framework) and PostgreSQL, featuring a multi-role admin hierarchy (Super Admin, Sub Admin, Broker, Hotel Manager), dynamic city management, real-time room availability checking, and a complete Ethiopian payment system.
3. **ERP System** — An Enterprise Resource Planning system with modules for HR management, inventory, sales, CRM, POS, and finance, built on CodeIgniter 4 with PostgreSQL.
4. **Oromia Communication Bureau Platform** — A Flutter-based government communication platform for the Oromia Communication Bureau, featuring localization, workflow automation, and institutional communication management.

The internship provided us with extensive hands-on experience in modern software development practices, including MVC architecture, RESTful API design, database design with PostgreSQL, JWT authentication, role-based access control, frontend development with Bootstrap 5, and mobile app development with Flutter. We gained practical skills in teamwork, project management, version control with Git, and professional communication in a real-world software development environment.

The internship ran from March 2 to June 2 in the Software Development section of Dallol Tech PLC. This report begins with the company background, then describes the internship experience, the benefits gained, the practical application of university knowledge, and the problems solved. It closes with a conclusion, recommendations, references, and appendices on the project file structure, database tables, and API endpoints.

TABLE OF CONTENTS

Cover Page....1
Declaration....2
Acknowledgements....3
Executive Summary....4
Table of Contents....5
Objective....7
1. Background of the Hosting Company....8
1.1 Brief History....8
1.2 Main Products and Services....8
1.3 Main Customers and End Users....9
1.4 Overall Organization and Work Flow....10
1.5 Overall Site Layout....11
2. Overall Internship Experience....13
2.1 How We Got Into the Company....13
2.2 Section of the Company We Worked In....13
2.3 Work Flow in the Section....14
2.4 Work Tasks We Executed....15
2.5 Procedures We Used....16
2.6 How Good We Were in Performing Our Tasks....18
2.7 Challenges and Measures Taken....18
3. Benefits Gained from the Internship....20
3.1 Practical Skills Improvement....20
3.2 Theoretical Knowledge Upgrading....21
3.3 Interpersonal Communication Skills....22
3.4 Team Playing Skills....22
3.5 Leadership Skills....22
3.6 Work Ethics....23
3.7 Entrepreneurship Skills....23
3.8 Challenges Faced....24
4. Practical Application of Knowledge....25
4.1 Database Design (Database Systems Course)....25
4.2 Web Programming (Web Programming Course)....25
4.3 Software Engineering (Software Engineering Course)....25
4.4 Object-Oriented Programming (OOP Course)....25

4.5 Data Structures and Algorithms 26
4.6 Human-Computer Interaction (HCI Course) 26
5. Problems Solved During the Internship 27
5.1 PostgreSQL Boolean Column Compatibility with CodeIgniter 4 27
5.2 Session Handling for Post-Auth Redirect 27
5.3 User Registration with Duplicate Email/Phone Detection 27
5.4 Dynamic City Management 28
5.5 Multi-Step Ethiopian Payment Flow 28
5.6 Room Availability Checking 29
5.7 Expert Recommendation System 29
6. Conclusion and Recommendations 30
6.1 Conclusion 30
6.2 Recommendations for the Company 30
7. References 32
8. Appendices 33
Appendix A: Project File Structure — Booking Website 33
Appendix B: Database Tables Overview 34
Appendix C: API Endpoints 34
Appendix D: Admin Panel URLs 35

OBJECTIVE

The primary objectives of this internship were:

1. **To gain practical experience** in a real-world software development environment, bridging the gap between theoretical knowledge acquired at the university and industry practice.
2. **To understand the software development lifecycle** in a professional setting, from requirements analysis to deployment and maintenance.
3. **To develop technical skills** in modern web and mobile application development, including PHP (CodeIgniter 4), Flutter (Dart), PostgreSQL database design, RESTful API development, and frontend technologies.
4. **To understand the organizational structure and workflow** of a technology company, including project management, quality assurance, and client communication.
5. **To contribute to real projects** that serve Ethiopian businesses and government institutions, gaining a sense of professional responsibility and work ethics.

To improve interpersonal and teamwork skills by collaborating with senior developers, QA engineers, and project managers in a professional environment

These objectives were pursued through direct participation in four projects with different technologies and clients. Working on the hotel booking mobile application and website, the ERP system, and the Oromia Communication Bureau platform gave us the chance to follow each stage of the software development lifecycle. It also exposed us to the way a technology company plans work, reviews code, tests features, and delivers products to private and government customers. The sections that follow show how each objective was met, and the Conclusion reflects on the results.

1. BACKGROUND OF THE HOSTING COMPANY

1.1 Brief History

Dallol Tech PLC is a modern software and IT solutions company based in Addis Ababa, Ethiopia. The company is inspired by the resilience of the Dallol region — one of the most extreme and geologically unique areas in the world, located in the Danakil Depression of northeastern Ethiopia. Just as the Dallol region represents extremes of nature and resilience, Dallol Tech PLC embodies resilience, innovation, and the drive to push technological boundaries in Africa.

Founded by **Mr. Abel Mulugeta**, Dallol Tech PLC has been operating for over **5 years** in the Ethiopian technology sector. The company began as a small software development team and has grown into a comprehensive technology solutions provider serving both private businesses and government institutions across Ethiopia.

The company received its **Certificate of Professional Competence** from the FDRE Ministry of Innovation and Technology (MOST) in 2020, acknowledging fulfillment of requirements for software development activities. Since then, the company has expanded its services and received multiple recognitions.

Today the company reports more than 120 happy clients, 82+ finished projects, and 120+ partnerships. Its recognitions include the 2020 Certificate of Professional Competence from MOST and an award in 2021 for the free website software and technology support it provided to Mekedonia Charity Association. These milestones show a company that combines commercial work with a commitment to society, and they made it a strong place to learn how professional software is built and delivered in Ethiopia.

1.2 Main Products and Services

Dallol Tech PLC offers a comprehensive suite of IT solutions organized into the following service pillars:

1. Managed Platforms
Scalable digital platforms tailored to organizational operations. This includes custom web applications, ERP systems, booking platforms, and enterprise management software.

2. Simulation & Training Systems
Hands-on systems and simulation programs for performance and safety, including driving simulator systems and school management platforms.

3. Security & Compliance
Security-first solutions with strong governance and reliability, including cybersecurity

consulting, secure payment integrations, and data protection services.

4. Custom Software Development

- Web application development using CodeIgniter 4, PHP, and modern frontend frameworks
- Mobile application development using Flutter (Dart) for Android and iOS
- Cloud solutions and database design with PostgreSQL
- API development and system integrations
- E-commerce and payment gateway integration

5. IT Consulting
Technology consulting for organizations undergoing digital transformation, including system architecture design, technology stack selection, and project planning.

6. Learning Hub
The company offers educational courses including:

- CodeIgniter 4 Complete Course (From Beginner to Advanced)
- Go (Golang) Complete Course
- PostgreSQL Course: From Zero to Database Hero

These service pillars are closely connected to the work we did as an intern. The Custom Software Development pillar is the closest match to our daily tasks, since we built web applications with CodeIgniter 4 and PostgreSQL and mobile applications with Flutter. The hotel booking platform and the ERP system belong to the Managed Platforms pillar, while the secure payment flows and role-based access control we implemented relate to the Security and Compliance pillar. The Learning Hub also shows the company's belief in sharing knowledge, which we saw in practice through the guidance we received from senior developers.

1.3 Main Customers and End Users

Dallol Tech PLC serves a diverse range of clients across Ethiopia:

Government Institutions:

- **Oromia Communication Bureau** — A large-scale government digital transformation platform to modernize communication infrastructure and automate institutional workflows.

Private Companies:

- **Yadi EV (Electric Vehicle)** — Contributed to fintech and enterprise digitalization projects, including secure payment gateway integrations and scalable transaction systems.
- **Rehoboth Drivers Training Center** — Delivered eSchool (school management) and driving-simulator systems.
- **Betelhem Yosef Telecom** — Short code development and integration with Ethio Telecom (9495).

Non-Profit Organizations:

- **Mekedonia Charity Association** — Provided free website software and technology support, recognized with an award in 2021.

General Clients:

- Hotels and hospitality businesses requiring booking platforms
- Educational institutions needing management systems
- Organizations requiring ERP and CRM solutions

The company has served over **120 happy clients** and completed **82+ finished projects** with **120+ partnerships** over its 5+ years of operation.

Several of these customers are directly connected to our internship. The Oromia Communication Bureau is the client of the government communication platform we worked on in the final phase of the internship, and the hotels and hospitality businesses listed above are the target users of the booking website and mobile application. The ERP system serves the kind of organizations that need HR, inventory, sales, CRM, POS, and finance management in one platform. Working on products for real customers made us understand that software must satisfy the needs of its end users, not only meet technical requirements.

1.4 Overall Organization and Work Flow

Dallol Tech PLC operates with a clear organizational structure designed to ensure efficient project delivery and quality control:

Leadership:

- **Abel Mulugeta** — Founder & CEO
- **Bezawit Nigussu** — Chief Finance Officer (CFO)

Technology Management:

- **Walelign Damtew** — Manager: Technology
- **Sisay Gebremedhin** — Development Manager: Software Engineering Expert
- **Leul Sewale, Habtamu Esubalew, Fikirte Shawul** — Senior Developers

Quality & Project Management:

- **Sintayehu Nigussie** — Manager: QA
- **Kidstemariam Asfaw** — Manager: Project
- **Mesay Alemayehu** — Manager: Business Strategy & Analysis

Data & Analytics:

- **Gedion Bula** — Business Support & Analytics Manager
- **Mikiyas Mesfin** — Data Science Specialist & Communication Liaison

Project Workflow: The company follows an agile-inspired development workflow:

1. **Requirements Gathering** — Client needs are analyzed by the Business Strategy team
2. **System Design** — Architecture and database design by the Technology team
3. **Development** — Implementation by the development team using MVC architecture
4. **Quality Assurance** — Testing and validation by the QA team
5. **Deployment** — Production release with ongoing support
6. **Maintenance** — Long-term partnership and optimization

This workflow matches the way we experienced project delivery during the internship. Requirements were discussed before implementation began, the database and architecture were designed before the code was written, and every feature passed through code review and QA testing before it was deployed. Because maintenance is treated as a long-term partnership, the team also kept improving existing products after release, which taught us that software work does not end at deployment.

1.5 Overall Site Layout

The company office is organized into functional zones:

- **Reception and Meeting Area** — Client meetings and initial consultations
- **Development Floor** — Open-plan workspace for developers with dual-monitor setups
- **QA Testing Area** — Dedicated testing environment with multiple devices and platforms
- **Project Management Office** — Planning and coordination space
- **Server Room** — On-premise servers and networking equipment
- **Conference Room** — Team meetings, presentations, and training sessions
- **Break Area** — Kitchen and rest facilities for staff

The development environment utilizes:

- **Version Control:** Git and GitHub for source code management
- **Development Frameworks:** CodeIgniter 4 (PHP), Flutter (Dart)
- **Database:** PostgreSQL for all production systems
- **Deployment:** Linux-based servers with Apache/Nginx
- **Testing:** Manual and automated testing with QA processes

This environment supported our work in a practical way. Git and GitHub let us work on features in separate branches without disturbing the code of other developers, and the shared PostgreSQL database standard meant that our schemas and queries followed the same conventions as the rest of the company. The dual-monitor setups on the development floor made it easier to write code while checking documentation or testing in the browser, and the QA testing area allowed our features to be checked on several devices before release.

2. OVERALL INTERNSHIP EXPERIENCE

2.1 How We Got Into the Company

We learned about Dallol Tech PLC through the university's internship placement program. After researching the company on their website (https://dalloltech.com/), we were impressed by their portfolio of real-world projects and their commitment to digital transformation in Ethiopia. We submitted our application along with our academic transcript and a cover letter expressing our interest in software development. Following a technical interview where we demonstrated our knowledge of PHP, database concepts, and basic mobile development, we were accepted as an intern in the development team.

Our internship began on March 2. At the start we were introduced to the development team, the company's project workflow, and the tools used in daily work, including Git, GitHub, CodeIgniter 4, Flutter, and PostgreSQL. Our mentor and the senior developers explained the structure of the projects we would join and the coding standards we were expected to follow. This introduction helped us begin contributing to real tasks quickly instead of spending a long time on setup.

2.2 Section of the Company We Worked In

During our internship, we were placed in the **Software Development** section under the supervision of the Technology Manager and senior developers. Our work was divided across four main projects, each representing a different aspect of modern software development:

Project 1: Hotel Booking Mobile Application (Flutter)

- Role: Mobile App Developer
- Technology: Flutter (Dart), RESTful API integration
- Duration: Initial phase of internship

Project 2: Hotel Booking Website (CodeIgniter 4 + PostgreSQL)

- Role: Full-Stack Web Developer
- Technology: PHP (CodeIgniter 4), PostgreSQL, Bootstrap 5, JavaScript
- Duration: Major portion of internship

Project 3: ERP System

- Role: Backend Developer
- Technology: CodeIgniter 4, PostgreSQL, Module-based architecture
- Duration: Middle phase of internship

Project 4: Oromia Communication Bureau Platform (Flutter)

- Role: Mobile App Developer
- Technology: Flutter, Localization, Government communication workflows
- Duration: Final phase of internship

Taken together, these four projects gave us a broad view of modern software development. The booking website and the ERP system taught us server-side development, database design, and secure authentication with CodeIgniter 4 and PostgreSQL. The two Flutter applications taught us mobile development, state management, API integration, and localization. Moving between web and mobile projects showed us how a single backend can serve several client applications and why consistent API design is important.

2.3 Work Flow in the Section

The development section followed a structured workflow:

1. **Daily Stand-up Meetings** — Each morning, the team discussed tasks, progress, and blockers.
2. **Task Assignment** — Tasks were assigned based on project priorities and individual skills. We received tasks through verbal assignment from our mentor and through project discussions.
3. **Development Phase** — we worked on our assigned tasks using:
    - Local development environment with PHP spark serve
    - PostgreSQL database for data storage
    - CodeIgniter 4 MVC architecture for web projects
    - Flutter framework for mobile projects
    - Git for version control
4. **Code Review** — Our code was reviewed by senior developers before merging.
5. **Testing** — QA team tested the features we developed on different devices and

scenarios.

6. **Deployment** — After passing QA, features were deployed to the production server.

This workflow helped us develop discipline in our own work. The daily stand-ups made us plan each day and report blockers early, the code reviews showed us how to improve readability and structure, and the QA feedback taught us to test our features carefully before handing them over. Each step also helped us learn how responsibilities are shared in a professional team and how the quality of a product depends on every stage of the process.

2.4 Work Tasks We Executed

During our internship, we executed a wide range of tasks across all four projects:

Hotel Booking Website Tasks:

- Designed and implemented the PostgreSQL database schema with 15+ tables including users, hotels, room_types, bookings, payments, reviews, wishlist, cities, admin_users, brokers, and hotel_managers
- Built the complete MVC architecture with CodeIgniter 4 — models, controllers, and views
- Implemented a multi-role admin hierarchy: Super Admin → Sub Admin → Broker → Hotel Manager, each with role-based access control
- Created dynamic city management — super admin can add any Ethiopian city (52 cities pre-loaded) and they appear instantly on the website
- Implemented real-time room availability checking — rooms show as "Available" or "Occupied for selected dates" based on existing bookings
- Built the complete booking flow: Hotel search → Room selection → Booking → Payment → Confirmation
- Implemented 12 Ethiopian payment methods (Telebirr, CBE Birr, Awash Bank, Dashen Bank, Amhara Bank, etc.)
- Created animated test payment forms with credit card preview, bank transfer forms, and mobile money forms
- Developed an expert recommendation system that suggests similar hotels based on city, price, rating, and user booking history

- Implemented JWT authentication for the mobile API endpoints
- Built the complete frontend with Bootstrap 5, custom CSS animations (hero slider, card carousels, scroll reveal), and JavaScript interactivity

Hotel Booking Mobile App Tasks:

- Developed the Flutter mobile application with screens for authentication, home, hotel search, hotel detail, booking, wishlist, and user profile
- Integrated the mobile app with the RESTful API built on the web backend
- Implemented state management with providers
- Designed the app UI following the booking website's visual language

ERP System Tasks:

- Worked on ERP modules including HR management, inventory control, sales, CRM, POS, and finance
- Built controllers for CRUD operations across multiple modules
- Designed database queries for reporting and dashboard analytics
- Implemented user authentication and role-based access

Oromia Communication Bureau Platform Tasks:

- Developed Flutter screens for the government communication platform
- Implemented localization for multiple languages (Afan Oromo, Amharic, English)
- Built communication workflow screens and institutional management features

Across the four projects, our tasks covered the full range of software development work: database design, backend logic, API development, frontend interfaces, mobile screens, authentication, payments, and reporting. Each task started from a requirement, moved through implementation and review, and ended with testing. Completing these tasks showed us how the individual parts of a system, such as the database, the API, the web interface, and the mobile app, depend on one another and have to be designed together.

2.5 Procedures We Used

Throughout our internship, we followed these development procedures:

Version Control:

- Used Git for all source code management

- Created branches for each feature
- Committed changes with descriptive commit messages
- Pushed code to GitHub repositories

Database Design:

- Designed normalized PostgreSQL schemas
- Used raw SQL queries to avoid ORM boolean quoting issues with PostgreSQL
- Created database triggers for auto-updating timestamps and ratings
- Implemented views for complex reporting queries

Development:

- Followed MVC (Model-View-Controller) architecture strictly
- Used CodeIgniter 4's routing system for clean URLs
- Implemented CSRF protection on all form submissions
- Used prepared statements and parameterized queries to prevent SQL injection
- Validated all user inputs both on frontend and backend

Testing:

- Manually tested all features using the browser
- Used curl commands to test API endpoints
- Verified database state after each operation
- Tested role-based access control for all admin roles

Deployment:

- Used `php spark serve` for local development
- Configured `.env` files for environment-specific settings
- Used dedicated database users with limited privileges

Following these procedures consistently was important because several of them protect the security and stability of the systems. Using prepared statements and CSRF protection reduces the risk of common web attacks, validating input on both the frontend and the backend prevents invalid data from reaching the database, and limiting database

privileges protects the production data. Version control with feature branches and clear commit messages also made our work easy for the team to review and trace.

2.6 How Good We Were in Performing Our Tasks

We demonstrated strong performance throughout our internship:

- **Fast Learning:** We quickly learned CodeIgniter 4's framework conventions, PostgreSQL-specific SQL syntax, and Flutter's widget system.
- **Problem Solving:** We independently resolved several technical challenges, including PostgreSQL boolean column compatibility with CodeIgniter 4's query builder, session handling issues, and payment flow edge cases.
- **Code Quality:** Our code followed the company's coding standards — clean indentation, descriptive variable names, proper error handling, and comprehensive comments.
- **Reliability:** We completed assigned tasks on time and communicated proactively when facing challenges.
- **Adaptability:** We successfully transitioned between web development (PHP/CodeIgniter 4) and mobile development (Flutter) as project needs changed.

Our mentor and the senior developers gave us feedback through code reviews and daily discussions, and we used this feedback to improve the quality of our work. As our tasks became more complex, we were trusted with larger responsibilities, including the full ownership of the booking website. We believe this progression shows that we performed well, learned from correction, and was able to meet the expectations of a professional development team.

2.7 Challenges and Measures Taken

Challenge 1: PostgreSQL Boolean Column Compatibility
CodeIgniter 4's query builder quoted PostgreSQL boolean values (TRUE/FALSE) as strings, causing database errors. We resolved this by using raw SQL queries with parameterized bindings instead of the query builder for boolean columns.

Challenge 2: Session Management
CodeIgniter 4 uses a different session API than Laravel. We initially used `session()->pull()` (a Laravel method) which doesn't exist in CI4. We resolved this by using `session()->get()` and `session()->remove()` separately.

Challenge 3: User Registration with Duplicate Detection

CI4's `is_unique` validation rule was causing unexpected errors during registration. We replaced it with manual database checks that provide clear, user-friendly error messages for duplicate emails and phone numbers.

Challenge 4: Payment System Integration
Ethiopian payment systems require specific handling for different methods (Telebirr, CBE Birr, bank transfers). We built a multi-step payment flow with test mode that simulates real payment processing without actual money transfer.

Challenge 5: Dynamic City Management
The requirement was that super admin could add any Ethiopian city and it would appear instantly on the website. We created a `cities` table with 52 pre-loaded Ethiopian cities and built dynamic dropdowns that pull from the database in real-time.

These challenges taught us that real projects rarely behave exactly as the documentation suggests. Each problem required us to read the error carefully, find its root cause, test a solution, and verify that the fix did not break other parts of the system. Dealing with framework-specific and database-specific behavior also showed us the value of reading official documentation, asking senior developers when we were stuck, and sharing the solution with the team afterward.

3. BENEFITS GAINED FROM THE INTERNSHIP

3.1 Practical Skills Improvement

During our internship at Dallol Tech PLC, we significantly improved our practical software development skills:

Backend Development:

- Mastered CodeIgniter 4 framework — routing, controllers, models, views, filters, helpers, and configuration
- Learned to design and implement complex PostgreSQL database schemas with proper normalization, foreign keys, triggers, and views
- Gained experience writing raw SQL queries with parameterized bindings for security and PostgreSQL compatibility
- Implemented JWT-based authentication for APIs and session-based authentication for web applications
- Built role-based access control systems with multiple user types

Frontend Development:

- Advanced our knowledge of Bootstrap 5 for responsive web design
- Learned to create custom CSS animations (keyframe animations, transitions, scroll reveal)
- Improved JavaScript skills — DOM manipulation, event handling, AJAX calls, form validation
- Built interactive UI components — image carousels, price calculators, payment method selectors, OTP input boxes

Mobile Development:

- Gained hands-on experience with Flutter framework
- Learned Dart programming language
- Implemented state management with providers
- Built RESTful API integration from mobile apps
- Worked with Flutter's widget system for complex UI layouts

Database Skills:

- Designed database schemas with 15+ related tables
- Wrote complex SQL queries with JOINs, subqueries, and aggregations
- Implemented database triggers for automatic timestamp updates and rating calculations
- Created database views for reporting dashboards
- Managed database users with appropriate privileges

In practice, these skills came together in the hotel booking system, where one feature often involved several layers. A single booking, for example, required a database design for rooms and bookings, a controller to process the request, a payment flow, and a user interface that showed the result clearly. Working across these layers improved both our technical depth and our ability to see a system as a whole.

3.2 Theoretical Knowledge Upgrading

Our internship reinforced and expanded our theoretical knowledge:

- **MVC Architecture:** We deeply understood the separation of concerns in Model-View-Controller pattern through real implementation
- **RESTful API Design:** We learned to design clean, stateless API endpoints with proper HTTP methods, status codes, and response formats
- **Database Normalization:** We applied normalization principles to real-world data with complex relationships
- **Authentication & Authorization:** We implemented both JWT (stateless) and session-based (stateful) authentication, understanding the trade-offs
- **Role-Based Access Control:** We designed a four-tier admin hierarchy with proper access filtering
- **Software Development Lifecycle:** We experienced the full cycle from requirements to deployment in a professional setting

Applying these concepts in real projects changed how we understand them. Topics that we had studied separately at the university, such as normalization, authentication, and software architecture, became connected parts of one working system. Seeing the consequences of design decisions, such as how a poorly chosen validation rule or query approach can cause errors for users, made the theory more meaningful and easier to remember.

3.3 Interpersonal Communication Skills

- **Daily Stand-ups:** We learned to communicate progress, blockers, and plans concisely in team meetings
- **Code Reviews:** We learned to accept and provide constructive feedback on code quality
- **Client Communication:** We observed how senior developers communicate technical concepts to non-technical clients
- **Written Communication:** We improved our technical writing through code comments, commit messages, and documentation

These communication skills are important for our future career. A developer must explain technical decisions to teammates, understand requirements from non-technical people, and document work so that others can continue it. The internship gave us regular practice in all of these, and we now feel more confident presenting our work and asking clear questions.

3.4 Team Playing Skills

- **Collaboration:** We worked alongside senior developers, QA engineers, and project managers, learning to coordinate tasks and share knowledge
- **Code Integration:** We learned to integrate our code with others' work using Git branches and pull requests
- **Knowledge Sharing:** We shared our solutions to PostgreSQL and CI4 compatibility issues with the team
- **Supporting Others:** We helped test features developed by other interns and provided feedback

Working in a team also taught us that the success of a project depends on shared responsibility. When one developer's code fails, other parts of the system can be affected, so communication, testing, and helping each other are as important as individual coding skill. These experiences prepared us for working in a professional software team after graduation.

3.5 Leadership Skills

- **Taking Ownership:** We took full responsibility for the booking website project, from database design to deployment
- **Decision Making:** We made architectural decisions (raw SQL vs query builder,

session handling approach) that affected the entire project

- **Mentoring:** We helped newer interns understand the codebase and development workflow
- **Initiative:** We proactively identified and fixed bugs, suggested improvements, and proposed new features

These leadership experiences gave us confidence in our ability to guide technical work. Making design decisions and explaining the reasons for them to the team taught us to think about long-term consequences, and helping newer interns taught us that explaining a concept is one of the best ways to understand it deeply.

3.6 Work Ethics

- **Professionalism:** We maintained regular working hours, met deadlines, and communicated proactively
- **Integrity:** We were honest about challenges and mistakes, and took responsibility for fixing them
- **Quality Focus:** We prioritized code quality, security, and user experience over speed
- **Respect:** We respected the company's confidentiality, coding standards, and team hierarchy
- **Continuous Learning:** We actively sought to learn new technologies and improve our skills

Maintaining these work ethics was important in an environment where our code was reviewed by others and used in real projects. Following the company's standards, protecting confidential information, and being honest about our progress built trust with our mentor and the team, and we intend to keep these habits throughout our professional career.

3.7 Entrepreneurship Skills

- **Problem Identification:** We learned to identify real-world problems that technology can solve (e.g., lack of online hotel booking in Ethiopia)
- **Solution Design:** We designed complete solutions from database to UI, considering business requirements
- **User-Centric Thinking:** We built features with the end user in mind — clear

error messages, intuitive flows, responsive design

- **Business Understanding:** We gained insight into how software products generate value for businesses and how payment systems work in Ethiopia

These experiences showed us that software engineering is also about creating value. The hotel booking platform, for example, solves a real problem for travelers and hotel owners in Ethiopia, and the Ethiopian payment methods make it practical for local users. This understanding encourages us to think about how we could build our own technology solutions for local needs in the future.

3.8 Challenges Faced

- **Technical Complexity:** The booking system involved complex relationships between hotels, rooms, bookings, payments, and multiple admin roles
- **Framework Learning Curve:** Learning CodeIgniter 4's specific conventions while avoiding common pitfalls took time
- **PostgreSQL Specifics:** PostgreSQL's strict type system and boolean handling differed from MySQL, requiring adaptation
- **Time Management:** Balancing work across four different projects with different technologies
- **Remote Collaboration:** Coordinating with team members on code integration and deployment

Although these challenges were demanding, they became useful learning experiences. The framework learning curve improved our self-study habits, the PostgreSQL differences taught us to understand a database deeply instead of relying on assumptions, and managing four projects improved our planning and prioritization. The measures we took to overcome each challenge are described in Section 2.7 and Section 5.

4. PRACTICAL APPLICATION OF KNOWLEDGE

During our internship, we applied the theoretical knowledge we gained at Debre Markos University to real-world projects at Dallol Tech PLC:

4.1 Database Design (Database Systems Course)

We applied normalization principles (1NF, 2NF, 3NF) to design the hotel booking database with 15+ tables. We used our knowledge of entity-relationship modeling to define relationships between users, hotels, rooms, bookings, payments, reviews, and admin roles. We implemented foreign key constraints, unique constraints, and CHECK constraints to maintain data integrity.

This work helped us understand why database design decisions matter in practice. A normalized schema made it easier to avoid duplicate data, keep bookings and payments consistent with hotels and users, and write reports without confusing joins.

4.2 Web Programming (Web Programming Course)

Our university courses in PHP and web programming directly applied to the CodeIgniter 4 projects. We used our knowledge of HTTP request/response cycles, form handling, session management, and URL routing to build the booking website's backend.

The web programming course was especially useful for understanding how the browser, the server, and the database communicate. Concepts such as GET and POST requests, form validation, cookies and sessions, and clean URLs appeared in almost every feature we built, and the course gave us the confidence to learn a new framework quickly.

4.3 Software Engineering (Software Engineering Course)

We applied software engineering principles including:

- **Modular Design:** Separating the system into Admin, API, and Web controllers
- **Design Patterns:** Using MVC, Singleton (database connection), and Factory patterns
- **Requirements Analysis:** Understanding client requirements for each project
- **Testing:** Manual and automated testing of all features

4.4 Object-Oriented Programming (OOP Course)

CodeIgniter 4 is built on OOP principles. We applied inheritance (extending Base controllers), encapsulation (protected/private properties), and polymorphism (overriding methods) throughout the codebase.

Understanding these object-oriented concepts also made it easier to read the framework's

own code and to organize our own classes, such as models, controllers, and helper methods, in a clean and reusable way.

4.5 Data Structures and Algorithms

We implemented efficient data structures for:

- Hotel search with multiple filters (city, price, rating, type)
- Room availability checking by querying overlapping date ranges
- Expert recommendation scoring algorithm based on multiple weighted factors
- Pagination for large result sets

4.6 Human-Computer Interaction (HCI Course)

We applied HCI principles to design intuitive user interfaces:

- Clear navigation with breadcrumbs and visual hierarchy
- Progressive disclosure (multi-step payment flow)
- Immediate feedback (loading spinners, toast notifications, animations)
- Error prevention (date validation, room selection before booking)
- Responsive design for different screen sizes

Together, these examples show how the courses we studied at Debre Markos University prepared us for professional work. The internship connected each subject to a real task and showed that theoretical knowledge becomes most valuable when it is applied to solve practical problems.

5. PROBLEMS SOLVED DURING THE INTERNSHIP

5.1 PostgreSQL Boolean Column Compatibility with CodeIgniter 4

Problem: CodeIgniter 4's query builder quoted PostgreSQL boolean values (TRUE/FALSE) as string literals ('TRUE'/'FALSE'), causing SQL errors like "column TRUE does not exist."

Solution: We replaced all query builder calls for boolean columns with raw SQL queries using parameterized bindings. For example, instead of `$builder->where('is_active', true)`, we used `$db->query("SELECT * FROM hotels WHERE is_active = TRUE")`. We also created helper methods like `pgBool()` to convert PostgreSQL's 't'/'f' string representations to PHP booleans.

Result: After this change, queries involving boolean columns worked correctly across the system, and the helper methods made the code easier to read and maintain. We also shared the approach with the team so that other developers working with CodeIgniter 4 and PostgreSQL could avoid the same error.

5.2 Session Handling for Post-Auth Redirect

Problem: When a user tried to book a hotel without being logged in, they needed to be redirected to login/register, then back to the hotel booking page. CodeIgniter 4 doesn't have Laravel's `session()->pull()` method.

Solution: We created a custom `_pullRedirect()` method that reads and removes the redirect URL from the session in two steps: `session()->get('post_auth_redirect')` followed by `session()->remove('post_auth_redirect')`. This preserved the booking intent across the authentication flow.

Result: Users could now begin a booking, log in or register when required, and return to the same hotel page without losing their intention. This improved the user experience and showed us how session data can be used to preserve the flow of a multi-step process.

5.3 User Registration with Duplicate Email/Phone Detection

Problem: CodeIgniter 4's `is_unique` validation rule caused unexpected errors during user registration, especially when a user with the same email or phone number already existed.

Solution: We replaced the framework's `is_unique` rule with manual database checks in the `createUser()` method. The method returns specific error codes ('email_exists',

'phone_exists') that the controller translates into clear, user-friendly messages with links to the login page.

Result: Visitors who try to register with an existing email or phone number now receive a clear message and a link to the login page instead of a confusing error. This solved the technical problem and also improved the usability of the registration process.

5.4 Dynamic City Management

Problem: The hotel search dropdown was hardcoded with a fixed list of cities. The client (super admin) needed to add any Ethiopian city and have it appear instantly on the website.

Solution: We created a `cities` table with 52 pre-loaded Ethiopian cities (including Debre Markos, Lalibela, Gambella, etc.) and built a `CityModel` that fetches active cities from the database. The home page, search dropdown, and admin panel all pull cities dynamically, so when a super admin adds a new city, it appears everywhere immediately.

Result: The super admin can now manage the available cities without changing any code, and the website updates automatically. This made the system more flexible for the client and showed us the value of storing configurable data in the database rather than in the source code.

5.5 Multi-Step Ethiopian Payment Flow

Problem: The booking system needed to support multiple Ethiopian payment methods (Telebirr, CBE Birr, Awash Bank, Dashen Bank, etc.) with different form requirements for each.

Solution: We built a `PaymentWebController` with a 4-step payment flow:

1. **Method Selection** — All 12 Ethiopian payment methods displayed in categorized cards
2. **Booking Summary** — Review page with hotel details, dates, and total price
3. **Payment Form** — Different forms for card (card number, expiry, CVV with animated preview), mobile (phone number with push notification preview), and bank (account number with bank card preview)
4. **Success Receipt** — Transaction ID, payment details, and booking reference

Each step has a progress indicator and the payment is simulated in test mode.

Result: The payment flow gave users a clear, guided path from choosing a method to

receiving a receipt. Because it runs in test mode, the team could demonstrate and test the full process safely, and the structure is ready to be connected to real payment gateways later.

5.6 Room Availability Checking

Problem: When a user selects dates for a hotel, the system needed to show which rooms are available and which are already booked for those dates.

Solution: We modified the `getRoomTypes()` method to accept check-in and check-out dates as parameters. For each room type, the method queries the `bookings` table for overlapping date ranges (using `check_in < ? AND check_out > ?`) and marks rooms as "Occupied for selected dates" if any confirmed or pending booking exists for those dates.

Result: Users can now see accurate room availability for their selected dates before making a booking, which reduces the risk of double booking and gives guests clearer information.

5.7 Expert Recommendation System

Problem: The hotel detail page needed to suggest similar hotels to users.

Solution: We built an `ExpertModel` with a scoring algorithm that evaluates candidate hotels based on:

- Same city (+5 points)
- Similar price range (+4 points)
- High rating (+3 points)
- Same hotel type (+2 points)
- Matching amenities (+1 per match, up to 5)
- User booking history preference (+4 points)
- Trending (high booking count in last 30 days) (+2 points)

The top 3 scored hotels are displayed as "You May Also Like" on the detail page.

Result: The recommendation feature helps guests discover other suitable hotels and keeps them engaged with the platform. It also showed us how simple, transparent scoring rules can provide useful personalization without a complicated machine-learning model.

6. CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

Our internship at Dallol Tech PLC was a transformative experience that bridged the gap between academic theory and professional practice. Over the course of our internship, we worked on four real-world software projects that serve Ethiopian businesses and government institutions, gaining hands-on experience in:

- Full-stack web development with CodeIgniter 4 and PostgreSQL
- Mobile app development with Flutter
- Database design and optimization
- RESTful API development
- Multi-role authentication and authorization
- Ethiopian payment system integration
- Government communication platforms

The internship reinforced our passion for software engineering and gave us confidence in our ability to contribute to real-world projects. We are grateful to Dallol Tech PLC for providing this opportunity and to Debre Markos University for preparing us with the theoretical foundation to succeed.

In addition to technical skills, the internship strengthened our professional habits. We learned to communicate in daily stand-ups, accept feedback from code reviews, work with colleagues of different roles, and take responsibility for our tasks from design through deployment. We now see more clearly how the theoretical courses at the university connect to the work of a software engineer, and we feel better prepared to start our career.

6.2 Recommendations for the Company

Based on our internship experience, we offer the following recommendations to Dallol Tech PLC:

1. **Adopt Automated Testing:** Implement PHPUnit tests for the CodeIgniter 4 projects to catch regressions early and improve code quality.
2. **API Documentation:** Generate API documentation using tools like Swagger/OpenAPI to improve communication between web and mobile development teams.

3. **Continuous Integration/Deployment:** Set up CI/CD pipelines using GitHub Actions to automate testing and deployment, reducing manual errors.
4. **PostgreSQL Connection Pooling:** Implement connection pooling (e.g., PgBouncer) for production databases to handle high-traffic scenarios.
5. **Real Payment Gateway Integration:** While the current test-mode payment system is excellent for development, integrate actual Telebirr and CBE Birr APIs for production use.
6. **Mobile App Publishing:** Complete and publish the Flutter mobile apps to Google Play Store and Apple App Store to reach wider audiences.
7. **Internship Program Enhancement:** Consider creating a structured internship curriculum with defined learning milestones and regular evaluation checkpoints.
8. **Documentation Standards:** Establish internal documentation standards for APIs, database schemas, and deployment procedures to facilitate knowledge transfer.

Recommendations for Debre Markos University: Based on our experience, we also recommend that the university strengthen the practical side of its software engineering courses. More laboratory work with PostgreSQL, a PHP framework such as CodeIgniter 4, and mobile development with Flutter would help students arrive at their internships better prepared. We also suggest that the university keep close contact with host companies during the internship, so that students' learning goals and the company's tasks stay aligned.

Final Remark: We believe that the recommendations in this report, if adopted, would benefit both the company and future interns, and we thank everyone who made this internship valuable.

7. REFERENCES

1. CodeIgniter 4 User Guide — https://codeigniter.com/userguide4/
2. PostgreSQL Documentation — https://www.postgresql.org/docs/
3. Flutter Documentation — https://docs.flutter.dev/
4. Bootstrap 5 Documentation — https://getbootstrap.com/docs/5.3/
5. Dallol Tech PLC Website — https://dalloltech.com/
6. Debre Markos University, Institute of Technology — Software Engineering Curriculum
7. Firebase PHP-JWT Library — https://github.com/firebase/php-jwt
8. PHP Manual — https://www.php.net/manual/en/
9. MDN Web Docs (JavaScript, CSS) — https://developer.mozilla.org/
10. Unsplash (Free Images) — https://unsplash.com/

8 APPENDICES

The appendices provide supporting technical material for the projects described in this report. Appendix A shows the file structure of the hotel booking website, Appendix B summarizes its database tables, Appendix C lists the REST API endpoints used by the mobile application, and Appendix D lists the URLs of the admin panel.

Appendix A: Project File Structure — Booking Website

The following structure shows how the CodeIgniter 4 booking website is organized according to the MVC pattern. Controllers are separated into API, Web, and Admin groups, filters handle JWT and session security, and models represent the database entities.

```
booking_website/
├── app/
│   ├── Config/          — App, Database, Routes, Filters,
Jwt, Paths
│   ├── Controllers/
│   │   ├── Api/         — Auth, Hotel, Booking, Guest,
Wishlist, Payment controllers
│   │   ├── Web/         — Home, HotelWeb, AuthWeb,
Dashboard, BookingWeb, PaymentWeb
│   │   └── Admin/       — SuperAdmin, SubAdmin, Broker,
HotelManager, AdminAuth
│   ├── Filters/         — JwtFilter, SessionFilter,
AdminFilter
│   ├── Helpers/         — jwt_helper, api_helper
│   ├── Models/          — User, Hotel, Booking, Review,
Wishlist, Payment, OTP, AdminUser, Broker, HotelManager,
Expert, City
│   └── Views/
│       ├── layouts/     — main.php, dashboard.php
│       ├── home/        — index.php (animated hero,
search, destinations, featured)
│       ├── hotels/      — list.php, detail.php (room
availability, booking form)
│       ├── auth/        — login.php, register.php,
otp.php, forgot.php
│       ├── dashboard/   — index, bookings, booking_detail,
wishlist, profile
│       ├── booking/     — confirm.php, payment/
│       ├── payment/     — method, summary, card, bank,
success
```

```
│       ├── admin/       — super, subadmin, broker,
manager, shared
│       └── errors/      — 404.php
├── database/
│   ├── schema.sql       — Main PostgreSQL schema (15
tables + seed data)
│   └── admin_schema.sql — Admin hierarchy schema (6 tables
+ super admin seed)
├── public/
│   ├── assets/css/app.css  — Full animated stylesheet
│   ├── assets/js/app.js    — Interactive JavaScript
│   └── index.php
├── composer.json
├── .env
└── START.sh
```

Appendix B: Database Tables Overview

The tables below show the database design of the booking system and the admin hierarchy, with the purpose of each table.

Booking Website Tables (15): | Table | Purpose | |-------|---------| | users | Guest accounts | | hotels | Hotel listings | | hotel_images | Hotel photo gallery | | hotel_amenities | Hotel features | | room_types | Room categories per hotel | | bookings | Reservation records | | payments | Payment transactions | | reviews | Guest reviews | | wishlist | Saved hotels | | otp_codes | OTP verification | | refresh_tokens | JWT refresh tokens | | cities | Ethiopian cities (52 entries) | | admin_users | Admin accounts | | brokers | Broker records | | hotel_managers | Hotel manager assignments |

Admin Hierarchy Tables (6): | Table | Purpose | |-------|---------| | admin_users | Super Admin, Sub Admin, Broker, Hotel Manager | | admin_city_assignments | City assignments to sub-admins | | brokers | Broker records with commission | | hotel_managers | Hotel-to-manager mappings | | broker_hotel_links | Broker-to-hotel links | | broker_messages | Inter-admin messaging |

Appendix C: API Endpoints

The following endpoints are used by the Flutter mobile application to communicate with the booking system. Endpoints marked JWT require an authenticated user.

Method	Endpoint	Purpose
POST	/api/v1/auth/guest/register	Register new guest
POST	/api/v1/auth/guest/login	Guest login

Method	Endpoint	Purpose
POST	/api/v1/auth/otp/send	Send OTP
POST	/api/v1/auth/otp/verify	Verify OTP
GET	/api/v1/hotels/search	Search hotels
GET	/api/v1/hotels/featured	Featured hotels
GET	/api/v1/hotels/{id}	Hotel detail
GET	/api/v1/hotels/{id}/reviews	Hotel reviews
POST	/api/v1/hotels/{id}/reviews	Post review (JWT)
GET	/api/v1/bookings	List bookings (JWT)
POST	/api/v1/bookings	Create booking (JWT)
GET	/api/v1/bookings/{id}	Booking detail (JWT)
POST	/api/v1/bookings/{id}/cancel	Cancel booking (JWT)
GET	/api/v1/guest/profile	Guest profile (JWT)
PUT	/api/v1/guest/profile	Update profile (JWT)
GET	/api/v1/wishlist	Wishlist (JWT)
POST	/api/v1/wishlist	Toggle wishlist (JWT)
POST	/api/v1/payments/initiate	Initiate payment (JWT)

Appendix D: Admin Panel URLs

These URLs belong to the admin panel and are protected by role-based access, so each admin role can open only the pages that match its permissions.

URL	Role	Purpose
/admin/login	All	Admin login
/admin/super/dashboard	Super Admin	System overview
/admin/super/cities	Super Admin	Manage cities
/admin/super/hotels	Super Admin	All hotels
/admin/super/hotels/add	Super Admin	Add hotel
/admin/super/sub-admins	Super Admin	Manage sub-admins
/admin/super/bookings	Super Admin	All bookings
/admin/super/users	Super Admin	All guests

URL	Role	Purpose
/admin/subadmin/dashboard	Sub Admin	City dashboard
/admin/subadmin/hotels	Sub Admin	Hotels in city
/admin/subadmin/brokers	Sub Admin	Manage brokers
/admin/broker/dashboard	Broker	Broker dashboard
/admin/broker/inbox	Broker	Messages
/admin/manager/dashboard	Hotel Manager	Hotel dashboard
/admin/manager/rooms	Hotel Manager	Manage rooms
/admin/manager/bookings	Hotel Manager	View bookings